

Prueba Técnica – Cloud Engineer Senior

Objetivo

Evaluar conocimientos prácticos y criterio técnico en:

- Arquitectura Cloud en AWS
- Networking y seguridad
- Kubernetes (EKS)
- Infraestructura como Código (Terraform)
- CI/CD con GitHub Actions
- Buenas prácticas de seguridad y observabilidad

La prueba busca evaluar cómo piensas y diseñas, no solo si el código funciona.

Reglas generales

- Todos los recursos deben crearse usando Terraform
- No exponer credenciales o secretos en texto plano
- Usar buenas prácticas de seguridad
- No usar acceso administrativo (AdministratorAccess)
- Todo debe estar versionado en un repositorio GitHub
- Asignar los siguientes tags a todos los recursos:
 - username
 - environment
 - project

Parte 1 – Diseño de la solución (OBLIGATORIA)

Entregables

1. Diagrama de arquitectura (PNG, PDF o draw.io)
2. Documento corto (máx. 1 página) explicando:
 - Diseño de la VPC
 - Flujo de red
 - Cómo EKS accede a AWS (S3 / ECR)
 - Cómo GitHub Actions se autentica en AWS
 - Decisiones técnicas tomadas

Parte 2 – Infraestructura AWS (Terraform)

Crear un proyecto Terraform que despliegue:

Networking

- VPC /16
- Subnets:
 - Públicas (ALB / NAT Gateway)
 - Privadas (EKS Nodes)
- Route Tables separadas
- Security Groups correctamente segmentados

Opcional (recomendado):

- VPC Endpoint para S3
- Cluster EKS privado

2 Amazon EKS

- Crear un cluster EKS funcional
- Node Group en subnets privadas
- Acceso a AWS usando **IAM Roles for Service Accounts (IRSA)**

3 Amazon S3

- Bucket para almacenar archivos de salida
- Carpeta obligatoria:
- s3://<bucket>/outputs/

4 Amazon ECR

- Repositorio ECR para las imágenes de los contenedores
- Imágenes versionadas correctamente

Parte 3 – Kubernetes (EKS)

Desplegar una solución en Kubernetes que cumpla lo siguiente:

Arquitectura

- Usar **CronJob** (o justificar otra opción)
- Un pod con dos contenedores (sidecar pattern)

Contenedor 1

- Script en Python o Bash
- Obtiene la IP privada del pod
- Crea un archivo .txt con:
 - IP privada
 - Timestamp de ejecución
- Nombre del archivo = timestamp
- Guarda el archivo en un volumen compartido

Contenedor 2

- Lee el archivo generado
- Lo sube al bucket S3 en la carpeta outputs/
- Autenticación exclusivamente vía IRSA

Logs

- Imprimir en logs:
- Timestamp de ejecución: <timestamp>
- Logs visibles en CloudWatch

Parte 4 – Seguridad e IAM

Implementar:

- IAM Role para GitHub Actions usando **OIDC**
- IAM Role para EKS Pods usando **IRSA**
- Políticas con mínimos permisos necesarios
- Prohibido usar Access Keys dentro de contenedores

Parte 5 – Terraform (estructura y calidad)

Se espera una estructura similar a:

```
terraform/  
├── modules/  
│   ├── vpc/  
│   ├── eks/  
│   ├── s3/  
│   └── ecr/  
├── envs/  
│   └── dev/  
│       ├── main.tf  
│       ├── variables.tf  
│       └── outputs.tf
```

Buenas prácticas esperadas:

- Uso de locals
- Validaciones en variables
- Outputs claros
- Backend remoto (S3 o Terraform Cloud)

Parte 6 – CI/CD con GitHub Actions

Feature Branch

Pipeline obligatorio:

- checkout
- terraform fmt
- terraform init
- terraform validate
- terraform plan

Rama develop

Pipeline obligatorio:

- checkout
- terraform init
- terraform apply

Reglas del repositorio

- No se permite push directo a develop
- Merge solo vía Pull Request
- Requisitos del PR:
 - Al menos 1 aprobación
 - Plan exitoso en feature branch

Autenticación AWS **solo vía OIDC**

Parte 7 – Observabilidad (Opcional)

Implementar al menos una:

- Logs estructurados en CloudWatch
- Métrica custom
- Dashboard básico

Parte 8 – Documentación (OBLIGATORIA)

Crear un README.md que incluya:

- Descripción general de la solución
- Diagrama de arquitectura
- Estructura del repositorio
- Cómo ejecutar Terraform
- Cómo funciona el flujo del CronJob
- Decisiones técnicas relevantes